

API Fundamentals System Design - Part 3

▼ 1. API definition

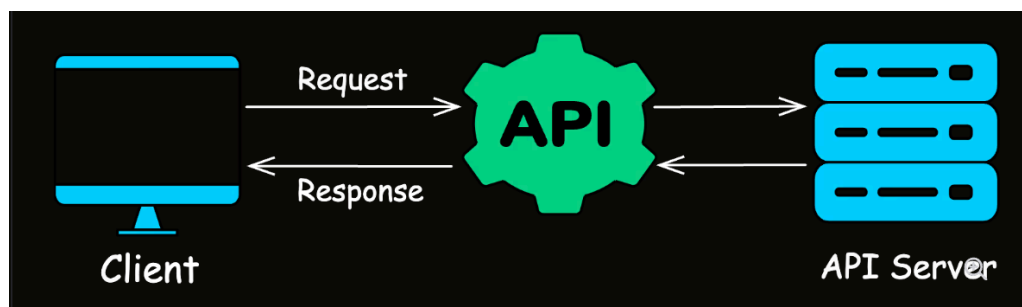
- APIs are a set of rules or protocols that allows two services or applications to communicate with each other.
- At its core, an API is a **bunch of code** that takes an **input** and gives you predictable **outputs**.
- You can think of an API as a **middleman** that enables applications to interact **without needing direct access to each other's code or database**.
- Almost every digital service that we use today - social media, ecommerce, online banking, ride hailing apps — all of them are powered by a bunch of APIs working together.

Building APIs

- When we build an API, we need to clearly define **what inputs the API accepts and what output it produces**, ensuring consistent behaviour.

API follows a request-response model:

- A client (such as a web app or mobile app) makes a request to an API.
- The API (hosted on an API server) processes the request, interacts with the necessary databases or services, and prepares a response.
- The API sends the response back to the client in a structured format (usually JSON or XML).



Inputs:

- Every API requires **specific types of inputs**. In some cases those inputs need to be in specific format. Passing incorrect data can result in errors. for ex:
The **Currency Exchange API** might need the input as `"USD_TO_EUR"` instead of `"usd to euro"`.

Outputs:

- Just as APIs require specific inputs, they also return well structured outputs. For ex: Google Maps API always returns coordinates in the same format.

```
{  
  "latitude": 40.6892,  
  "longitude": -74.0445  
}
```

If they can't find the location, it provides appropriate error response explaining why?

```
{  
  "error": "Invalid address format",  
  "code": 400  
}
```

▼ How APIs power modern applications?

The apps that we use today — Gmail, Instagram, Uber etc are essentially a collection of APIs with a polished UI on top. Most applications follow the backend / frontend architecture.

The backend consists of APIs that handles **data processing, business logic, and communication with databases**.

The frontend is a GUI that interacts with these APIs making applications user-friendly and accessible **without requiring users to write code**.

For ex: let's take a real world example of Uber. So before the Uber app existed as a sleek, user friendly experience, the company first built the core APIs that powers the ride hailing services.

- Finding Nearby Drivers
- Calculating Fares & Routes
- Process Payment
- Real-Time tracking

- Matching Riders & Drivers

These APIs run on Uber's servers forming the backend infrastructure. Everytime we request a ride, track our driver, or make a payment these backend APIs handles the request.

Backend engineers are responsible for optimizing these APIs, improving ride matching algorithms, securing transactions, and ensuring a smooth experience for millions of users.

Backend handles all the complex logic, but it's not feasible for normal users to interact with those APIs. That's why we build a frontend (user interface) on top of these APIs, allowing users to interact with the system visually & intuitively.

For ex: when we enter our pickup & destination address, the frontend sends an API request to find nearby drivers. Once the trip is complete, the frontend may call the process payment API to display the receipt.

▼ Types of APIs

Public APIs: these are often free APIs provided by companies to encourage 3rd party developers to integrate their services and build new applications on top of them. For ex: weather Api

Private APIs (also known as internal APIs) —

- These are designed exclusively for internal use within an organisation. Unlike public APIs these are not accessible to external developers and are used to facilitate seamless communication between different internal systems within a company.
- For ex: Amazon, when we place an order, it feels like a single system processes our request. In reality, multiple internal APIs (order processing, inventory, payment, logistics etc..) work together behind the scenes to fulfill our order efficiently. Each of these APIs operates independently, but they communicate through well defined protocols to ensure smooth & efficient process.
- Internal APIs allows companies to break down their applications into smaller, manageable services making it easier to scale.
- Developers can also reuse internal APIs across different projects reducing duplication and speeding up development.

Code Interfaces / Library APIs / Programming APIs

These APIs don't connect different applications; instead, they provide predefined functions within a programming language or framework to make development easier. For ex: Python's built-in list API. When working with lists, Python provides a set of **built-in functions (methods) to manipulate data**.

These APIs abstract away the complexity so that developers can focus on building solutions rather than reinventing the wheel.

```
numbers = [5, 3, 8, 1, 4] numbers.sort() # API call to sort the list
fruits = ["apple", "banana"]
fruits.append("orange") # API call to add an element
fruits.pop() # API call to remove the last element
```

▼ API communication Methods

1. REST (**R**epresentational **S**tate **T**ransfer)

- a. Most widely used API communication method
- b. Lightweight, Stateless and Scalable — making it perfect for web services and mobile application
- c. REST APIs use HTTP methods like (GET, POST, PUT, DELETE) to perform operations.
- d. They are based on resources, where each resource is accessed through a URL (endpoint)
- e. The API follows the client-server model, meaning the client sends a request, the server processes it and sends a response.

2. SOAP (SImple Object Access Protocol)

- a. Older communication method that **relies on XML-based messaging**.
- b. Unlike REST, which is lightweight, SOAP is **more structured and secure**, making it ideal for banking, healthcare, and enterprise applications.

- c. SOAP Messages are sent using XML format and require a WSDL (Web Services Description Language) file, which defines the APIs available functions and request structure.

3. GraphQL

- a. Serves as an alternative to REST, that allows clients to request exactly the data they need, making it more efficient.
- b. Unlike REST, which requires multiple API calls to fetch related data, GraphQL can **fetch all necessary data in a single request**.
- c. Unlike predefined endpoints, GraphQL exposes a **single API endpoint**, and the client sends queries to request specific fields.

For ex: Fetching a user's profile and their recent posts in a single request.

```
{
  user(id: 123) {
    name
    email
    posts {
      title
      likes
    }
  }
}
```

```
# Response JSON format
{
  "data": {
    "user": {
      "name": "Alice",
      "email": "alice@example.com",
      "posts": [
        {
          "title": "Hello World",
          "likes": 100
        },
        {
          "title": "GraphQL is Amazing!",

```

```
    "likes": 200
  }
]
}
```

4. gRPC (Google Remote Procedure Call)

1. It's a high performance API communication method that uses protocol buffers (Protobuf) instead of JSON or XML, making it faster and more efficient.
2. gRPC uses binary data format instead of text-based formats. reducing payload size.
3. It also supports **bidirectional streaming**, meaning the client and server can send data at the same time.

▼ How to use an API?

1. First step is to find the right API
2. Go thru the API documentation, understand available endpoints, authentication mechanism and response formats
3. Get API key — most api's require authentication to prevent unauthorized access and manage usage limits. Below are few common authentication methods.
 - a. API Key — a unique key provided by the API service
 - b. OAuth 2.0 — Secure login via Google, Github, etc.
 - c. JWT — Token based authentication
 - d. Basic Authentication — Username + Password (Base 64 encoded)
4. Test the API using postman or cURL
 - a. cURL allows you to test api's from the command line. For ex:

```
curl -X GET "https://api.openweathermap.org/data/3.0/weather?q=New+York&appid=YOUR_API_KEY"
```
5. Write code to call the API
 - a. now that we've tested the API, its time to integrate it into our application.

```

import requests

API_KEY = "YOUR_API_KEY"
CITY = "New York"

url = f"https://api.openweathermap.org/data/3.0/weather?q={CITY}&appid={API_KEY}"

response = requests.get(url)

if response.status_code == 200:
    data = response.json()
    temperature = data['main']['temp']
    print(f"Current temperature in {CITY}: {temperature}°C")
else:
    print(f"Error retrieving data: Status code {response.status_code}")

```

- `requests.get(url)` – Sends an API request.
- `response.json()` – Converts response to JSON.
- `if response.status_code == 200` – Checks if the request was successful.

6. Handle Errors and Rate Limits

- APIs don't always return perfect responses. We need to handle
 - invalid inputs
 - authentication errors
 - rate limits

```

import requests

API_KEY = "YOUR_API_KEY"
CITY = "New York"

url = f"https://api.openweathermap.org/data/3.0/weather?q={CITY}&appid={API_KEY}"

```

```

response = requests.get(url)

if response.status_code == 200:
    data = response.json()
    weather_description = data['weather'][0]['description']
    print(f"Current weather in {CITY}: {weather_description}")
elif response.status_code == 401:
    print("Error: Invalid API key")
elif response.status_code == 404:
    print("Error: City not found")
else:
    print(f"Unexpected error occurred: Status code {response.status_code}")

```

▼ 2. API Gateways

As applications grow in size, the number of APIs increases too. Without the right tools and infrastructure, managing these APIs can quickly become a challenge.

This is where **API Gateway** comes into play.

An API Gateway acts as a **central server** that sits between clients (e.g., browsers, mobile apps) and backend services.

Instead of clients interacting with multiple microservices directly, they send their requests to the API Gateway. The gateway processes these requests, enforces security, and forwards them to the appropriate microservices.

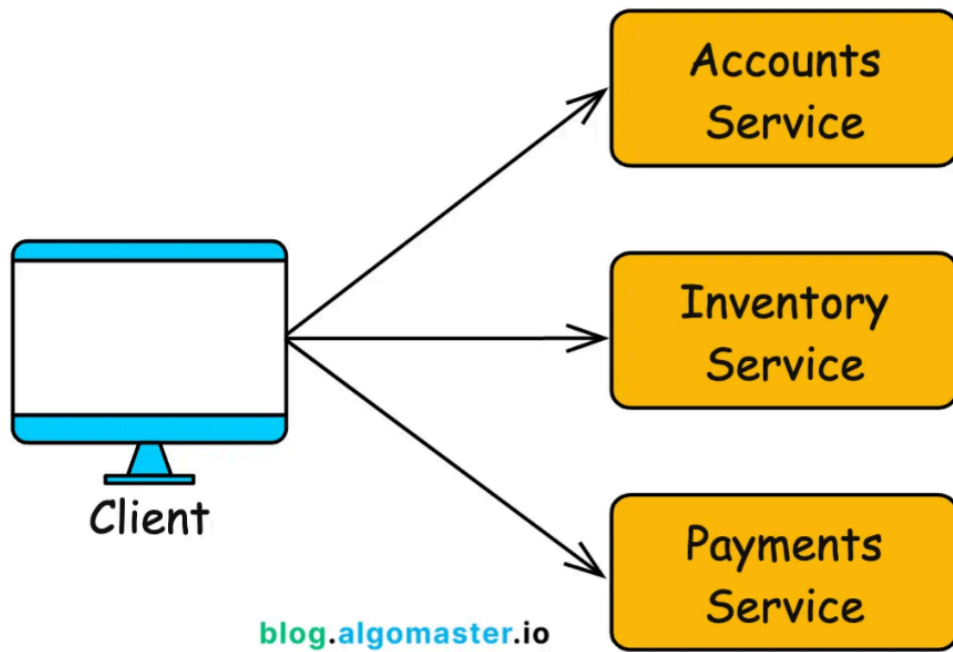
▼ Why do we need an API gateway?

Modern applications, especially those built using microservices architecture, have multiple backend services managing different functionalities.

For example, in an e-commerce service:

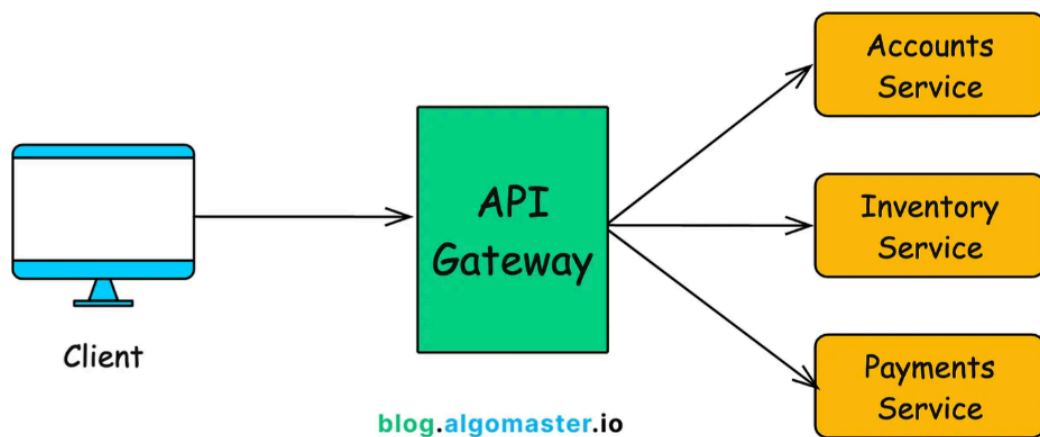
- One service handles **user accounts**.
- Another handles **payments**.
- Another manages **product inventory**

Without an API Gateway:



- Clients would need to know the location and details of all backend services.
- Developers would need to manage authentication, rate limiting, and security for each service individually.

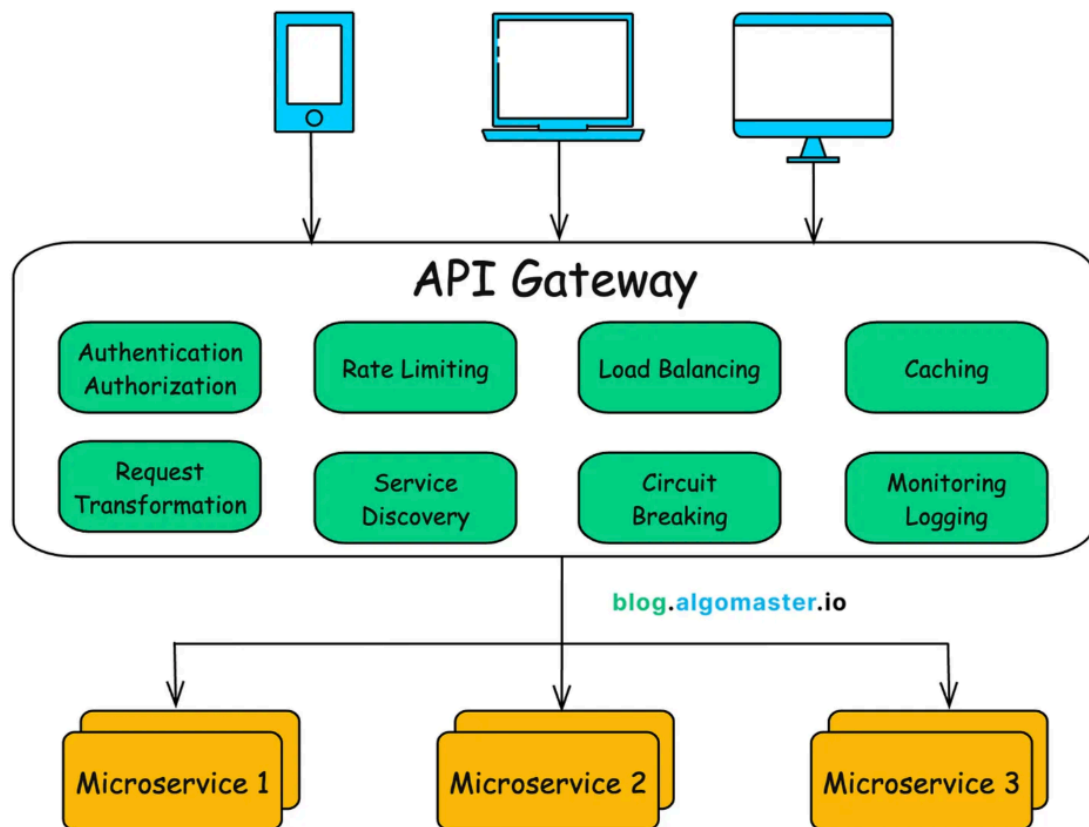
With an API Gateway:



- Clients send all requests to one place – the API Gateway.

- The API Gateway takes care of routing, authentication, security, and other operational tasks, simplifying both client interactions and backend management.

▼ Core Features of an API gateway



▼ 1. Authentication and Authorization

API Gateway secures the backend systems by ensuring only authorized users and clients can access backend services.

It handles tasks like:

- **Authentication:** Verifying the identity of the client using tokens (e.g., OAuth, JWT), API keys, or certificates.
- **Authorization:** Checking the client's permissions to access specific services or resources.

By centralizing these tasks, the API gateway eliminates the need for individual services to handle authentication, reducing redundancy

▼ 2. Rate Limiting

To prevent abuse and ensure fair usage of resources, most API Gateways implement **rate limiting**.

This feature:

- Controls the frequency of requests a client can make within a given timeframe.
- Protects backend services from being overwhelmed by excessive traffic or potential denial-of-service (DoS) attacks.

For example, a public API might allow a maximum of 100 requests per minute per user. If a client exceeds this limit, the API Gateway will block additional requests until the rate resets.

▼ 3. Load Balancing

High-traffic applications rely on **load balancing** to distribute incoming requests evenly across multiple instances of a service.

The API Gateway can:

- Redirect requests to healthy service instances while avoiding ones that are down or overloaded.
- Use algorithms like round-robin, least connections, or weighted distribution to manage traffic intelligently.

▼ 4. Caching

To improve response times and reduce the strain on backend services, most API Gateways provide **caching**.

They temporarily store frequently requested data, such as:

- Responses to commonly accessed endpoints (e.g., product catalogs or weather data).
- Static resources like images or metadata.

Caching helps in reducing latency and enhancing user experience while lowering the operational cost of backend services.

▼ 5. Request Transformation

In systems with diverse clients and backend services, **request transformation** is essential for compatibility.

An API Gateway can:

- Modify the structure or format of incoming requests to match the backend service requirements.
- Transform responses before sending them back to the client, ensuring they meet the client's expectations.

For instance, it might convert XML responses from a legacy service into JSON for modern frontend applications.

▼ 6. Service Discovery

Modern systems often involve microservices that scale dynamically.

The **service discovery** feature of an API Gateway dynamically identifies the appropriate backend service instance to handle each request.

This ensures seamless request routing even in environments where services frequently scale up or down.

▼ 7. Circuit Breaking

Circuit breaking is a mechanism that temporarily stops sending requests to a backend service when it detects persistent failures, such as:

- Slow responses or timeouts.
- Server errors (e.g., HTTP 500 status codes).
- High latency or unavailability of a service.

The API Gateway continuously monitors the health and performance of backend services and uses circuit breaking to block requests to a failing service.

▼ 8. Logging and Monitoring

API Gateways provide robust **monitoring and logging** capabilities to track and analyze system behavior.

These capabilities include:

- Logging detailed information about each request, such as source, destination, and response time.
- Collecting metrics like request rates, error rates, and latency.

This data helps system administrators detect anomalies, troubleshoot issues, and optimize the system's performance. Many API Gateways also integrate with monitoring tools like Prometheus, Grafana, or AWS CloudWatch.

▼ How Does an API Gateway Work?

Imagine that we're placing an order on Zomato. When we tap "Place Order", our phone makes an API request. But instead of talking directly to various backend services, it communicates with an API gateway first.

▼ 1. Request Reception

When you tap "Place Order," the app sends a request to the **API Gateway**, asking it to process our order. This request includes things like:

- our user_id,
- authentication tokens
- selected restaurant and menu items
- delivery address
- payment method

The API gateway receives the request as the single entry point to the backend

▼ 2. Request Validation

Before forwarding the request, the API gateway validates it to ensure:

- The required parameters or headers are present.
- The data is in the correct format (e.g., JSON).
- The request conforms to the expected structure or schema.

If any information is missing or incorrect, the gateway immediately rejects the request and notifies the app with an appropriate error message.

```
// Example of initial request handling
app.post('/api/v1/orders', async (req, res) => {
  // Check if request has required headers
  if (!req.headers['content-type'].includes('application/j
```

```
son')) {
    return res.status(400).send('Invalid content type');
}
// Continue processing...
});
```

▼ 3: Authentication & Authorization

The gateway now identifies your identity and permissions to ensure only legitimate users can place orders.

- It forwards your authentication token (e.g., OAuth or JWT) to an identity provider to confirm your identity.
- It checks your permissions to ensure you're authorized to use the app for placing an order.

If authentication or authorization fails, the API Gateway sends a `401 Unauthorized` or `403 Forbidden` error back to the app.

```
const authenticateRequest = async (req) => {
  // Extract JWT token from header
  const token = req.headers.authorization?.split(' ')[1];

  // Verify token and get user details
  const user = await verifyToken(token);

  // Check if user has permission to place orders
  return user.permissions.includes('place_orders');
};
```

▼ 4: Rate Limiting

To prevent abuse, the API Gateway checks how many requests you've made recently. For example:

- If you've made 10 "Place Order" requests in the last minute (maybe by accident), the gateway might block additional requests temporarily and return `429 Too Many Requests` response.

This ensures the system remains stable and fair for all users specially during traffic spikes or malicious attacks, such as distributed denial-of-service (DDoS) attempts.

```

const checkRateLimit = async (userId) => {
  const key = `rate_limit:order:${userId}`;
  const current = await redis.incr(key);

  // If first request in window, set expiry
  if (current === 1) {
    await redis.expire(key, 60); // 1 minute window
  }

  return current <= 10; // Allow 10 order requests per minute
};

```

▼ 5: Request Transformation (if needed)

If any of these backend services require specific data formats or additional details, the API Gateway transforms the request.

For example:

- The app sends the delivery address in plain text, but the Delivery Service expects GPS coordinates. The API Gateway converts the address into coordinates before forwarding the request.

```

const transformRequest = async (originalRequest) => {
  const address = originalRequest.deliveryAddress;

  // Convert address to GPS coordinates using a geocoding API
  const coordinates = await getCoordinatesFromAddress(address);

  if (!coordinates) {
    throw new Error('Failed to fetch GPS coordinates');
  }

  // Transform the request for the Delivery Service
  return {
    orderId: originalRequest.orderId,
    customerName: originalRequest.customerName,
  };
};

```

```

    deliveryLocation: {
      latitude: coordinates.lat,
      longitude: coordinates.lng
    },
    deliveryInstructions: originalRequest.instructions ||
  ""
};
};
};

```

▼ 6: Request Routing

The API Gateway now needs to coordinate several backend services to process your order.

Using **service discovery**, it identifies different services:

- **Order Service:** To create a new order record.
- **Inventory Service:** To check if the restaurant has your selected items available.
- **Payment Service:** To process your payment.
- **Delivery Service:** To assign a delivery driver to your order.

The gateway dynamically routes the request to these services using a **load balancing** algorithm, ensuring it connects to available and healthy service instances.

```

const routeRequest = async (req, serviceType) => {
  // Get service registry
  const services = await serviceDiscovery.getServices(serviceType);

  // Select instance
  const targetService = selectServiceInstance(services);

  // Forward request
  return await axios.post(
    `${targetService.url}/api/orders`,
    req.body,
    { headers: req.headers }
  );
}

```



```
);  
};
```

▼ 7: Response Handling

Once the API Gateway receives the response(s) from the backend service(s), it performs the following tasks:

- **Transformation:** Adjusts the response format or structure to match the client's requirements.
- **Caching (Optional):** Stores the response temporarily for frequently accessed data, reducing future latency.

Finally, the API Gateway sends the processed response back to the client in a format they can easily understand.

```
const handleResponse = async (serviceResponse) => {  
  // Transform response if needed  
  const transformedResponse = {  
    orderId: serviceResponse.order_reference,  
    estimatedDelivery: serviceResponse.eta,  
    status: serviceResponse.current_status  
  };  
  
  // Cache response if applicable  
  if (serviceResponse.cacheable) {  
    await cacheResponse(  
      transformedResponse.orderId,  
      transformedResponse  
    );  
  }  
  
  return transformedResponse;  
};
```

▼ 8: Logging & Monitoring

API gateway has robust monitoring and logging capabilities. So throughout this process, the gateway records important metrics to track each request.

```
const logRequest = async (req, res, timing) => {
  await logger.log({
    timestamp: new Date(),
    path: req.path,
    method: req.method,
    responseTime: timing,
    statusCode: res.statusCode,
    userId: req.user?.id
  });
};
```

▼ 3. REST vs GraphQL

APIs are the backbone of modern applications, acting as the bridge between **client applications** and **backend servers**.

Among the many API design choices, **REST** and **GraphQL** have emerged as two dominant approaches. Both offer powerful ways to retrieve and manipulate data, but they are built on fundamentally different philosophies.

▼ REST

1. It's a time-tested architectural style, provides structured APIs around fixed endpoints and HTTP methods, making it intuitive and widely adopted.
2. REST is not a protocol but rather a set of guiding principles that leverages the existing HTTP protocol to enable communication between clients & servers.
3. At its core, REST is built around **resources**. Each resource (such as a user, order, or product) is uniquely identified by a **URL** and clients interact with these resources using a **fixed set of HTTP methods**. For ex:
 - **GET** → Retrieve a resource (e.g., `GET /api/users/123` to fetch user data).
 - **POST** → Create a new resource (e.g., `POST /api/users` to add a new user).
 - **PUT/PATCH** → Update an existing resource (e.g., `PUT /api/users/123` to update user details).
 - **DELETE** → Remove a resource (e.g., `DELETE /api/users/123` to delete a user).
4. REST APIs typically return JSON response and use HTTP status codes to communicate the outcome of each request. For ex:
 - **200 OK** → Success

- **201 Created** → Resource successfully created
- **400 Bad Request** → Client error (e.g., missing required fields)
- **404 Not Found** → Requested resource does not exist
- **500 Internal Server Error** → Unexpected server issue

Key Benefits of REST

- **Simplicity and Intuitive Design:** The resource based models aligns well with most business domains.
- **Statelessness:** Each and every request is independent of one another and contains all the information it needs to complete it. This is what makes REST scalable across distributed systems
- **Cacheability:** REST leverages HTTP's caching mechanisms to improve response times.
- **Mature Ecosystem:** REST enjoys robust tooling, documentation and developer familiarity.

Drawbacks of REST

1. **Overfetching:** REST endpoints often return more data than needed, leading to unnecessary bandwidth usage. For ex: if a mobile app only needs a user's name and email, but the API response includes additional fields like address, phone number, and metadata, it results in **wasted bandwidth**.
2. **Underfetching:** Scenarios of underfetching are also possible where an API doesn't return all related data. In such cases, clients need to make multiple API calls to retrieve all required info. For ex: to get user details and their posts, a client might have to make 2 GET api calls
 - GET /api/users/123 (fetch user)
 - GET /api/users/123/posts (fetch user's posts)
3. **Versioning:** This is another major challenge with REST APIs. As API evolves, maintaining backward compatibility becomes really difficult. That is why REST APIs often require versioned URLs like /v1/users, /v2/ users adding maintainance overhead.
4. **Rigid Response Structure:** The server defines how / what data is to be returned and clients must adapt to it — even if they only need a subset of the data.

▼ GraphQL

For years, **REST** was the de facto standard for building APIs. However, as applications grew more complex, REST began to show limitations—especially in scenarios where clients needed fine-grained control over the data they fetched. To address these challenges facebook introduced GraphQL offering a more flexible and efficient approach to data retrieval.

▼ How GraphQL works?

Unlike REST, which organizes APIs around **fixed endpoints and HTTP methods**, GraphQL is a **query language** that allows clients to request exactly the data they need—nothing more, nothing less.

A **single GraphQL endpoint** (`/graphql`) replaces multiple REST endpoints, allowing clients to structure their own queries instead of relying on predefined responses. For ex: below query fetches a specific user's firstName, email, profile url and posts all within a single request.

```
query {
  user(id: 123){
    firstName
    email
    profile{
      url
    }
    posts{
      title
      publishedDate
    }
  }
}
```

So the way it works is, GraphQL aggregates the data from multiple services and returns precisely the requested data. Thus it solves the overfetching & underfetching problem that exists with REST

▼ 3 Core Functionalities of GraphQL

1. Queries to fetch data:

- a. Similar to GET requests in REST, GraphQL queries allow clients to request specific fields of data offering them full control over what they

retrieve.

2. Mutations to modify data:

- a. This is equivalent to POST, PUT, PATCH, or DELETE in REST. It is used to create, update or delete resources.

Example: Creating a new post

```
mutation {  
  createPost(title: "GraphQL vs REST", content: "GraphQL solves many of REST's l  
imitations...", publishedDate: "2025-03-10") {  
    id  
    title  
    content  
  }  
}
```

3. Subscriptions offer realtime updates:

- a. Unlike REST which requires polling or websockets for real time updates, GraphQL subscriptions enable clients to listen for changes and receive updates automatically when data is modified. Ideal for chat applications, live feeds, stock market updates, notifications etc.

b. Example: Listening for new posts

```
subscription {  
  newPost {  
    title  
    content  
    author {  
      name  
    }  
  }  
}
```

Whenever a **new post is created**, all subscribed clients will **receive instant updates**.

▼ Key Benefits of GraphQL

1. Precise data fetching

- a. Clients can request only the fields they need, reducing over-fetching and under-fetching.

2. Single Request for Multiple Resources:

- a. Related data can be retrieved in one request, solving REST's `n+1` query problem.

3. Strong Typing:

- a. GraphQL APIs use a schema to define available data, making them easier to explore and document

4. Real-time Data with Subscriptions:

- a. GraphQL natively supports real-time data updates through subscriptions, enabling clients to receive automatic notifications whenever data changes on the server.

5. API Evolution Without Versioning:

- a. New fields can be added without breaking existing queries, avoiding REST-style `/v1`, `/v2` versioning issues

▼ Drawbacks of GraphQL

1. Complex Setup & Tooling:

- a. Unlike REST, which can be used with basic HTTP clients (cURL, browsers), GraphQL requires a GraphQL server, schema, and resolvers.

2. Caching challenges:

- a. REST APIs leverage HTTP caching (e.g., browser caching, CDNs), but GraphQL queries use POST requests, making caching trickier.

3. Increased Server Load:

- a. Since clients can request arbitrary amounts of data, GraphQL APIs must be carefully optimized to prevent performance issues.

4. Security Risks:

- a. Unoptimized queries (e.g., deeply nested requests) can lead to costly database scans, increasing the risk of denial-of-service (DoS) attacks.

5. Performance Risks

For ex: let's say a mobile app introduces a new feature that unexpectedly triggers a full table scan on a critical database table.

With REST, this scenario is less likely because API endpoints are predefined and developers control what data is exposed.

With GraphQL, the client constructs the query which could inadvertently request massive amounts of data. If a poorly designed query is executed on a high traffic service, it could bring down the entire db.

To mitigate this, GraphQL APIs require strict rate limiting, depth restrictions & cost analysis mechanisms — adding additional complexity to the implementation

▼ GraphQL vs REST differences

They both rely on HTTP requests and responses, but they differ in how they structure and deliver response. For ex: REST centers around resources (each resource is uniquely identified by a URL) whereas GraphQL centers around a schema that defines the types of data available.

In REST, the API implmenter decides which data/field is included in the response. For ex: If a client requests a blog post, the API might also return related **author details**, even if they aren't needed. With GraphQL the client decides what to fetch. This makes GraphQL more flexible, but also introduces challenges in caching and performance optimization.

▼ Which one should we pick?

There is no one-size fits all solution. REST remains a great choice for simple APIs, while GraphQL is powerful for complex applications with varying data needs.

Ultimately it's not about which is better, but which is better for our specific needs.

Use REST if

- Our API is simple and doesn't require flexible queries.
- When we need caching benefits from HTTP.
- When integrating with 3rd Party services

Use GraphQL if

- We need flexible and efficient data fetching.
- If our API serves multiple clients (mobile, web, IOTs) with different data needs.
- If we need real time updates — GraphQL subscriptions
- If we want to avoid API versioning issues.

NOTE: In fact, we can even use both REST & GraphQL. So these are not mutually exclusive and many orgs implement a hybrid approach to get the best of both worlds.

▼ 4. Websockets

What are Websockets and why are they used?

1. a communication protocol which is used to build real time features by establishing a 2way connection between the client and server.

For ex: an online multiplayer game where the leaderboard **updates instantly** as players score points, showing **real-time rankings** of all players.

This instantaneous update feels seamless and keeps us engaged. But how does it actually work? — The magic behind this real time experience is often powered by websockets.

2. Web sockets enable **bidirectional, full-duplex** communication between a client and a server over a **single TCP connection**.

Unlike the traditional HTTP protocol, where the client sends a request to the server and waits for a response, WebSockets allow both the client and server to send messages to each other independently and continuously after the connection is established.

▼ How do websockets work?

The WebSocket connection starts with a standard HTTP request from the client to the server.

However, instead of completing the request and closing the connection, the server responds with an **HTTP 101** status code, indicating that the protocol is switching to WebSockets.

After this handshake, a WebSocket connection is established, and both the client and server can send messages to each other over the open connection.

Step by step process

1. The client initiates the connection by sending a standard HTTP GET request with an "Upgrade" header set to "websocket".
2. Now, if the server supports websockets and accepts the request, it responds with a special 101 status code, indicating that the protocol will be changed to websocket.
3. Once the handshake is complete, the websocket connection is established. This connection remain open until explicitly closed by either the client or server.

4. Both client & server can now send & receive messages in real time. These messages are sent in small packets called **frames**. These frames have minimal overhead compared to traditional HTTP requests.
5. The connection can be closed at any time by either the client or server, typically with a "close" frame indicating the reason for closure.

▼ Why are websockets used / Key Benefits of Websockets?

1. **Real time updates:** websockets enable instant data transmission, making them perfect for applications that require real time updates like gaming apps, live chat, financial trading platforms etc.
2. **Bidirectional communication:** Both client and server can initiate communication, allowing for more dynamic and interactive applications.
3. **Lower Overhead:** Websockets Frames have lower overhead (small headers as small as 2bytes) compared to the traditional HTTP requests.
4. **Reduced Latency:** Since the connection is persistent, there's no need to establish a new connection for each message, significantly reducing latency.
5. **Efficient Resource Usage:** WebSockets are more efficient than traditional polling techniques, as they don't require the client to continuously ask the server for updates.

▼ Websockets vs Polling vs Long Polling vs HTTP

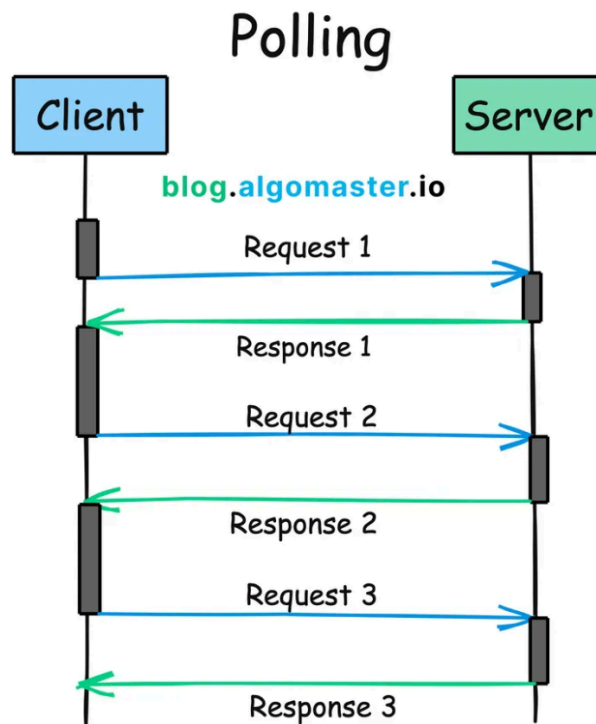
HTTP:

- HTTP follows the request-response model. Client sends a request, the server processes it and return the response back to client, closing the connection afterward.
- HTTP is stateless which means each request is independent of the other requests and they typically have all the info they need contained in that request.
- HTTP is not suitable for real time communication because each interaction requires a new request and thus it has a higher latency compared to websockets making it not a good choice for real time updates.

Polling:

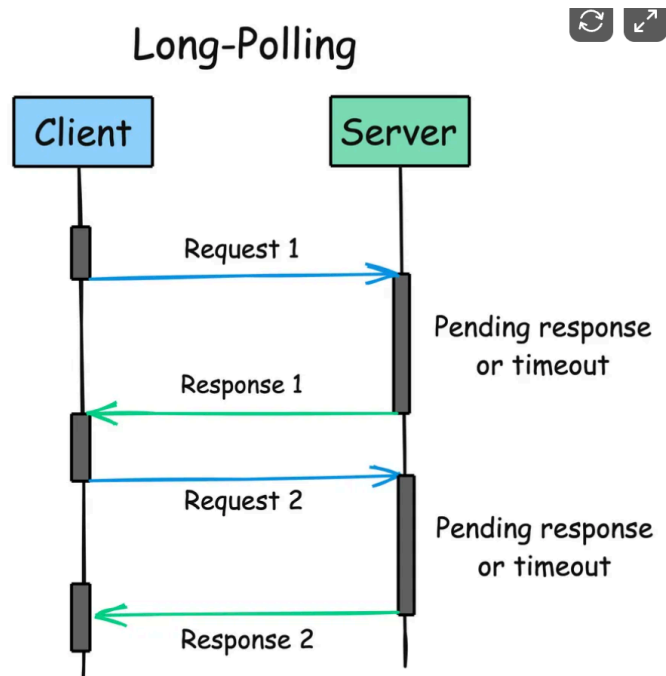
- Here, the client repeatedly sends requests to the server at fixed intervals to check for updates. Now while this can simulate real time updates, it is highly inefficient because majority of our requests will return no data.

- Polling can introduce slight delays because the requests are sent periodically. (at fixed intervals)



Long Polling:

- In long polling, the client sends a request and the server holds the connection open until it has data to send or request times out eventually closing the connection and then the client immediately sends a new request.
- So this approach reduces the frequency of requests but still suffers from higher latency compared to websockets (because it keeps the connection open until it has data to send or request times out).
- Also it can lead to resource exhaustion on the server as it has to manage multiple open connections.



4. Websockets:

- a. These are bidirectional, unlike HTTP, polling and long polling allowing 2 way communication.
- b. Because the connection remains open, thus it offers lower latency bcoz data can be sent and received with minimal delays.
- c. Websockets are highly efficient in terms of resource & bandwidth usage.

▼ Challenges and Considerations with Websockets

While WebSockets offer numerous benefits, there are some challenges to consider:

- **Scalability Concerns:** Managing a large number of WebSocket connections can be challenging. We might need to use load balancers and distributed websocket servers to handle large scale connections.
- **Fallback Mechanism:** Not all clients or networks support WebSockets, which can lead to connectivity issues. We need to implement fallback mechanisms like long-polling for clients that cannot establish WebSocket connections.
- **Proxy Servers:** Some proxy servers don't support WebSocket connections and certain firewalls may block them.

- **Network Reliability:** WebSockets rely on a persistent connection, which can be disrupted by network issues. Implementing **reconnection strategies** and **heartbeat mechanisms** (regular ping/pong messages) can help maintain the connection's stability and detect when a connection has been lost.
- **Security:** WebSockets are vulnerable to attacks such as Cross-Site WebSocket Hijacking and Distributed Denial of Service (DDoS) attacks. Implement **secure WebSocket connections (wss://)**, authenticate users, and validate input to protect against common vulnerabilities.

▼ Real world Applications of Websockets

1. Real-Time Collaboration Tools:

Applications like Google Docs may use WebSockets to enable multiple users to edit a document simultaneously. Changes made by one user are instantly reflected for all others, creating a seamless collaborative experience.

2. Real-Time Chat Applications

This is one of the most popular usecase of websockets — real time chat apps. These platforms like (slack, whatsapp, messenger etc) use websockets to deliver messages instantly which enables real time conversations.

3. Real Time Notifications

Social media platforms use websockets to push real time notifications to users when they receive a message, like or comment. Instead of the client constantly checking for new notifications, the server pushes the updates to the client as soon as they occur.

4. Multiplayer Online Games:

In online multiplayer games, low latency is crucial for seamless experience. Websockets provides the real time updates between gamers and servers, ensuring that all players see the same game state simultaneously.

5. Stock Markets

websockets are widely used in trading platforms for real time market data such as stock prices.

6. IOT applications:

In IOT applications, devices often need to communicate with a server in real time.

7. Live Streaming and Broadcasting:

While the actual video streaming typically uses other protocols, WebSockets can be used for real-time chat, viewer counts, and other interactive features during live broadcasts.

▼ Implementing WebSockets

below code snippets provide server side implementation in node js and client side implementation in javascript

▼ 5. Webhooks

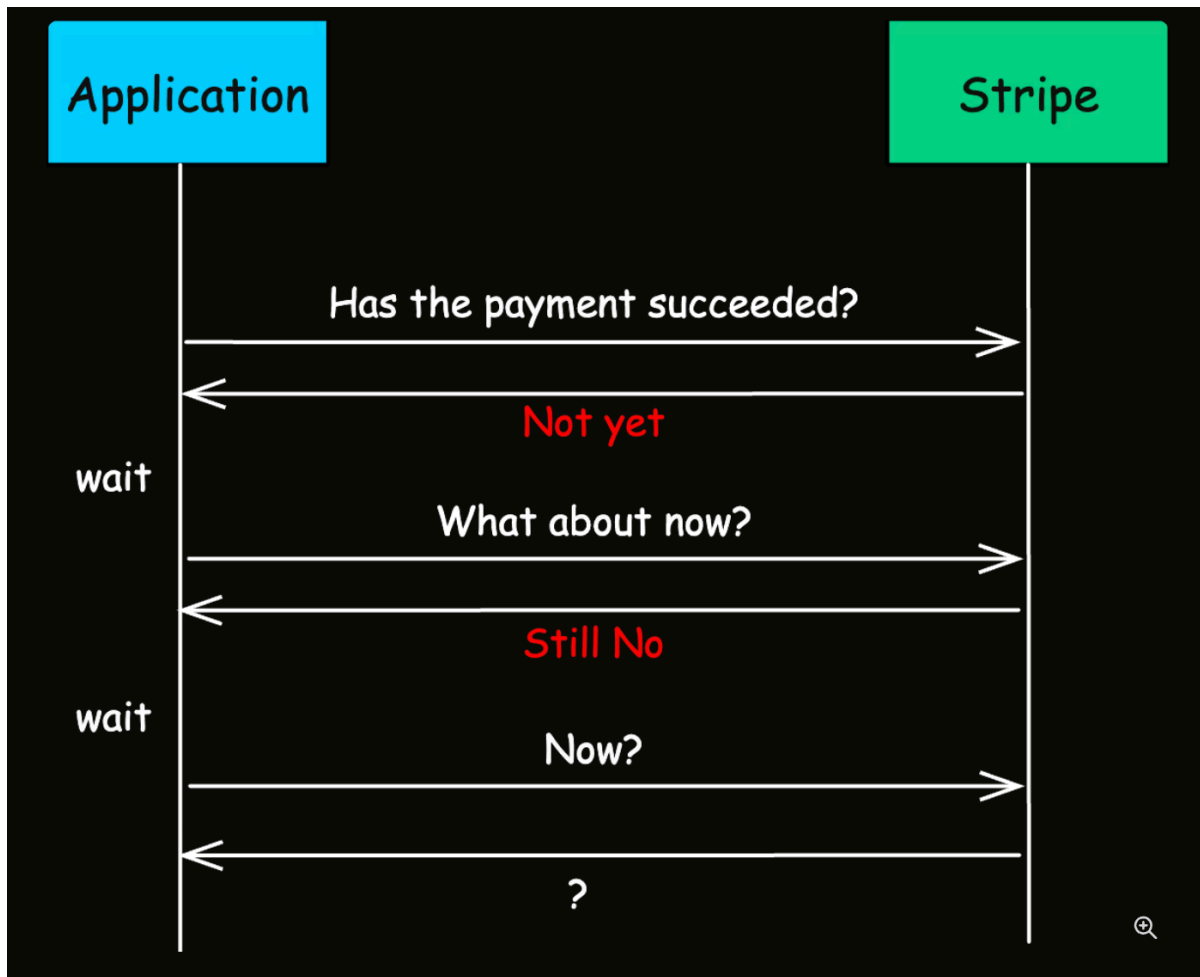
Imagine you're building an e-commerce platform and using an external payment processor like Stripe to collect payments from users.

Once a user completes a payment, your system needs to:

- Mark the order as paid
- Send an invoice
- Notify the warehouse to start packing

But here's the challenge: Stripe operates on its own infrastructure. Your system doesn't control it. So how do you know **instantly** when a payment goes through?

A naive solution would be to keep asking Stripe every few seconds:



This is known as **polling**.

Now imagine doing this for **every order** on a site like Amazon.

It just doesn't scale and wastes server resources.

Instead of your app repeatedly asking, **what if Stripe could just tell you** when the payment succeeds?

That's what **webhooks** do.

▼ 1. What is a Webhook?

A webhook is a simple way for one system (provider) to notify another system (receiver) in real time when an event happens using an HTTP request.

Instead of one system asking repeatedly if something happened, the other system simply pushes the data as soon as the event is triggered.

▼ 2. How Webhooks work?

At a high level, webhooks work through **registration, triggering, and delivery**.

Let's walk through a real example to see what actually happens under the hood.

Example: GitHub Webhook to Your App

Let's say you've built a system that needs to react when someone opens a pull request (PR) in a GitHub repository — maybe to kick off a CI/CD pipeline, send a Slack notification, or run automated tests.

Here's how the webhook flow works step-by-step:

Step 1: You Register a Webhook

- You go to GitHub → Settings → Webhooks
- You provide a **webhook URL** (e.g., `https://myapp.com/github-webhook`)
- You choose the **events** you care about, like `pull_request`, `push`, or `issue_comment`

At this point, GitHub knows where to send data whenever those events occur.

Step 2: GitHub Monitors Events

- GitHub monitors its internal event stream for your repository
- When someone opens a new pull request, the event gets logged internally

Step 3: GitHub Sends a POST Request

- GitHub prepares a **JSON payload** with details about the event: who opened the PR, which branch, repo metadata, etc.
- It makes a `POST` request to your webhook URL:

Step 4: Your Server Processes the Event

- Your app receives the request
- You verify the signature (to ensure it really came from GitHub)
- You process the payload — maybe enqueue it in a job queue, log it, or trigger business logic
- You respond with an HTTP `200 OK` to acknowledge receipt

▼ 3. Anatomy of a Webhook Request

To integrate with webhooks successfully, you need to understand **what's actually being sent** to your server when the event occurs.

Most webhook providers use the **HTTP POST method** to send event data to your server.

- Why POST? Because it allows the payload to be sent in the request body, which can include structured data (usually JSON) describing what happened.
- Rarely, some services allow `GET`, `PUT`, or even `PATCH`, but `POST` is the de facto standard for webhook delivery.

Request Headers

The headers in a webhook request often include **metadata** and **security-related information**.

Common headers include:

- **Content-Type:** Usually `application/json` (or sometimes `application/x-www-form-urlencoded`)
- **User-Agent:** Identifies the sender (e.g., `Stripe/1.0`, `GitHub-Hookshot`)
- **X-Event-Type:** Describes what type of event occurred (`payment_intent.succeeded`, `pull_request`, etc.)
- **X-Signature / X-Hub-Signature:** A hash of the payload using a secret key (HMAC) used to verify the request authenticity
- **X-Request-ID:** Unique ID for the webhook event (useful for logging/debugging)

Request Body

The body of a webhook request typically contains **event data** in **JSON format**. This data includes what happened, when it happened, and which user/resource it's related to.

Example: GitHub PR Event

```
{
  "action": "opened",
  "number": 42,
  "pull_request": {
```



```

    "id": 11223344,
    "title": "Add login validation",
    "user": {
      "login": "octocat"
    },
    "created_at": "2025-04-21T12:00:00Z"
  },
  "repository": {
    "name": "awesome-project",
    "full_name": "octocat/awesome-project"
  },
  "sender": {
    "login": "octocat"
  }
}

```

Example: Stripe Payment Intent Succeeded

```

{
  "id": "evt_1PoJkD2eZvKYlo2Cm0JbvwD9",
  "object": "event",
  "api_version": "2024-08-01",
  "created": 1713681701,
  "data": {
    "object": {
      "id": "pi_3JhdNe2eZvKYlo2C1IqojYg9",
      "object": "payment_intent",
      "amount": 2000,
      "currency": "usd",
      "status": "succeeded"
    }
  },
  "livemode": false,
  "type": "payment_intent.succeeded"
}

```

▼ 4. Setting up a Webhook Receiver

Once you've registered your webhook URL with a provider like GitHub or Stripe, the next step is to **build a robust endpoint on your server** that can receive and process these events reliably.

This sounds simple — just receive a POST request, right?

But in production, things get tricky:

- Webhooks might arrive **multiple times** or **out of order**
- Your server might be **under load**, or briefly **unavailable**
- **Malicious actors** might try to spoof requests

Let's break down how to setup your webhook receiver the *right* way.

4.1 Basic Setup

Create a Dedicated Endpoint

Start by exposing a simple HTTP endpoint like:

```
POST /webhook
```

This endpoint should:

- Accept only POST requests
- Accept only JSON (via `Content-Type`)
- Be accessible over HTTPS

Make It Idempotent

Webhook events can be **retried**, **duplicated**, or even **replayed** by providers. Your handler must ensure that processing the same event **multiple times** has no side effects.

How?

- Every webhook event includes a **unique ID** (e.g., `evt_1234` in Stripe, `delivery_id` in GitHub)
- Store a record of processed IDs in a database or cache
- Before processing, check if the event ID has already been handled

This ensures **exactly-once processing**, even if you receive the same event more than once.

Handle HTTP Status Codes Properly

- Return `200 OK` after successful handling
- Return `400 Bad Request` if the payload is invalid or malformed
- Avoid 5xx errors unless something truly failed because most providers **retry** on 5xx

4.2 Security Considerations

Webhooks are public endpoints. That means **anyone on the internet can POST to them**. You must validate incoming requests.

Verify the Signature (HMAC)

Most providers include a **cryptographic signature** of the payload, using a shared secret. This ensures the payload came from the real source.

Example (Stripe):

- You configure a webhook secret: `whsec_abc123`
- Stripe signs the payload using HMAC-SHA256
- You verify the signature on your end:

This prevents **spoofed or forged webhooks**.

Whitelist Source IPs (Optional)

Some providers publish static IP ranges from which webhooks are sent. You can optionally block all other IPs to tighten access.

| Downside: This adds DevOps complexity and can break things if IPs change.

Don't Expose Sensitive Data

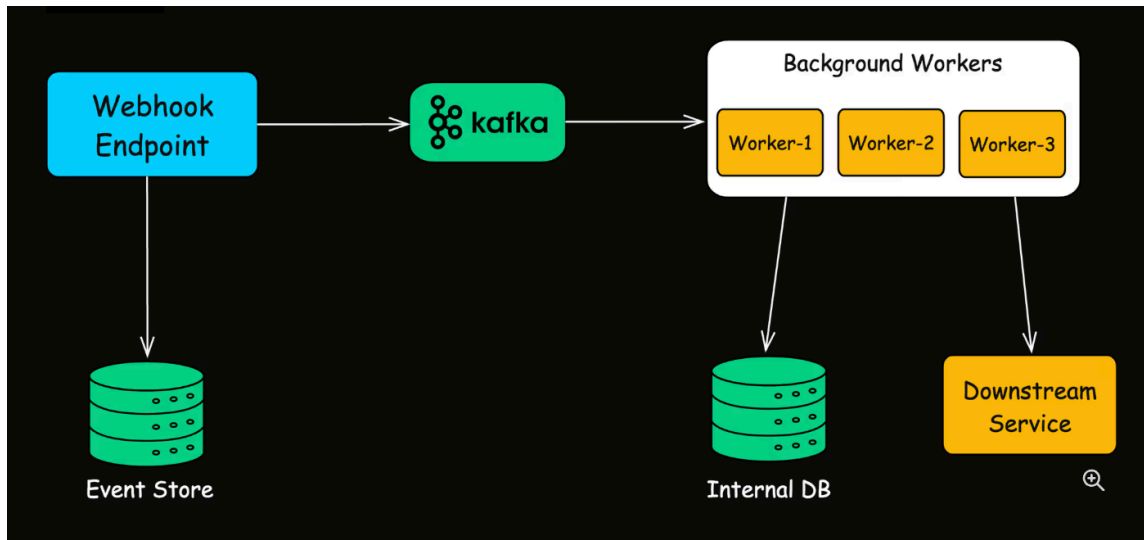
- Don't log full webhook payloads in plaintext
- Don't return stack traces or internal errors in responses
- Don't include secrets or tokens in the response body

▼ 5. Designing a Scalable Webhook Infrastructure

As your application grows and starts receiving thousands or even millions of webhook events daily, a simple synchronous handler won't be enough.

To make your webhook system **reliable, fault-tolerant, and scalable**, you need a design that's **asynchronous, decoupled, and observable**.

Let's walk through how to build a production-grade webhook processing pipeline.



▼ 5.1 Queue Incoming Requests

Don't do heavy processing inside your webhook endpoint.

Instead:

- Parse and validate the request
- Verify its authenticity (signature, event ID, etc.)
- Immediately enqueue the event into a **message queue** like:
- **Kafka** (for high-throughput event pipelines)
- **RabbitMQ** (lightweight and flexible)
- **AWS SQS** (serverless and easy to manage)

This way, your server can return a fast **200 OK**, while actual processing happens in the background.

▼ 5.2 Store Events for Audit & Replay

To ensure traceability and recovery, store every incoming event in a database:

- Raw payload
- Event ID and type

- Source info (e.g., IP address, headers)
- Status (`queued` , `processed` , `failed`)
- Timestamps

Use a durable data store like PostgreSQL, MongoDB, or DynamoDB.

┆ This gives you the ability to **replay failed events** and **debug issues**.

▼ 5.3 Process with Asynchronous Workers

Background workers (or worker pools) pulls events from the queue and perform actual business logic:

- Deduplicate the event using eventID
- Run validations
- Update internal databases or call downstream services
- Trigger notifications or workflows

Why use workers?

- You can scale them horizontally
- You gain control over load and processing rate
- You isolate failures from the webhook endpoint

▼ 5.4 Retry with Backoff

Sometimes, event processing fails temporarily due to a database issue, timeout, or an unavailable downstream API.

Your workers should **retry** those events automatically, using a **backoff strategy**:

- **Exponential** (recommended): retry with increasing delays (1s, 2s, 4s...)
- **Linear**: retry every N seconds (e.g., retry every 30s)
- **Jitter**: add randomness to avoid all retries happening at once

┆ Always limit retries and log failures after the final attempt.

▼ 5.5 Use a Dead Letter Queue (DLQ)

If a webhook consistently fails after multiple retries, don't keep retrying forever.

Instead, send it to a **Dead Letter Queue**, a special holding queue for problematic events.

From there, you can:

- Trigger alerts
- Investigate the root cause
- Retry manually after fixing the issue

This prevents bad events from clogging your pipeline and ensures no event is lost silently.

▼ 5.6 Add Observability: Logs, Metrics & Alerts

You can't fix what you can't see. Add observability at every stage of the pipeline.

Track metrics like:

- Total webhook events received per hour
- Success vs. failure rate
- Processing latency
- Queue length
- Retry count and DLQ event count

Set alerts for:

- Spikes in failure rate
- Sudden drop in incoming events
- Growing queue sizes (indicating backlog)

Popular observability tools:

- **Prometheus + Grafana** (open-source)
- **Datadog, CloudWatch, New Relic** (cloud-based)
- **ELK Stack** (Elasticsearch + Logstash + Kibana)

With proper observability, you'll catch issues early before they impact your users.

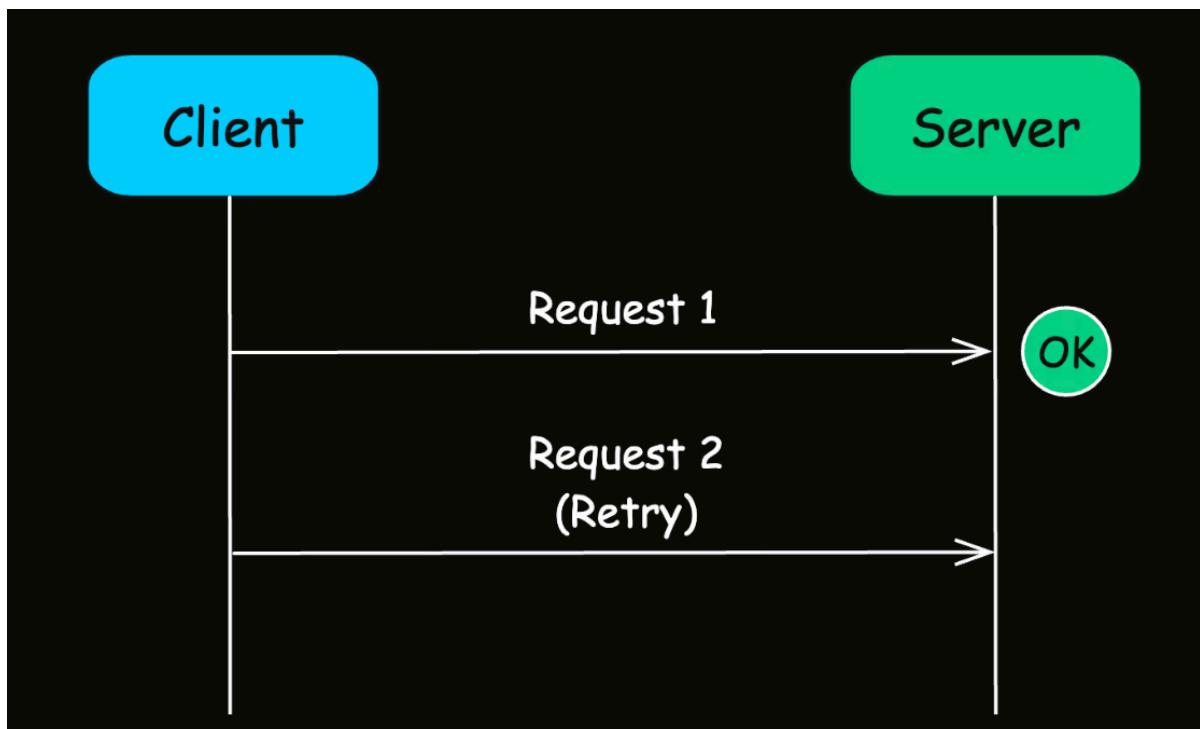
▼ 6. Idempotency

Imagine you're making a **purchase** from an online store.

You hit "**pay**" but the screen freezes, and you're unsure if the payment went through.

So, you **refresh** the page and **try again**.

Behind the scenes, how does the system ensure you aren't accidentally **charged twice**?



This scenario highlights a common problem in distributed systems: **handling repeated operations gracefully**.

The solution to this problem lies in the concept of **idempotency**.

▼ 1. What is Idempotency?

In mathematics, an operation is idempotent if applying it multiple times produces the same result as applying it once.

For example, the absolute value function is idempotent: $||-5|| = |-5| = 5$.

Idempotency is a property of certain operations whereby executing the same operation multiple times produces the same result as executing it once.

For example: If a request to delete an item is idempotent—all requests after the first will have no impact.

In programming, setting a value is idempotent, while incrementing a value is not.

```
Idempotent: user.status = 'active'
```

```
Not Idempotent: user.login_count += 1
```

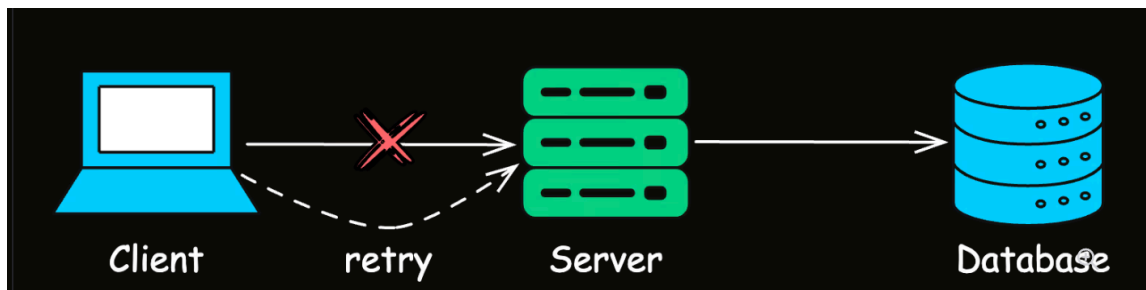
Some operations are naturally idempotent.

```
UPDATE users SET status = 'active' WHERE id = 123;
```

No matter how many times you run this, the result remains the same.

▼ 2. Why Idempotency Matters

Distributed systems often require **fault tolerance** to ensure high availability. When a network issue causes a **timeout** or an **error**, the client might **retry** the request.



If the system handles retries without idempotency, every retry could change the system's state unpredictably.

By designing operations to be idempotent, engineers create a buffer against unexpected behaviors caused by retries.

This "safety net" prevents repeated attempts from distorting the outcome, ensuring stability and reliability.

▼ 3. Strategies to Implement Idempotency

▼ 1. Unique Request Identifiers

One of the simplest techniques to achieve idempotency is by attaching a **unique identifier**, often called an **idempotency key** to each request.

When a client makes a request, it generates a **unique ID** that the server uses to track the request. If the server receives a request with the same ID later, it knows it's a duplicate and discards it.

Example: A payment service could require every transaction request to include a unique ID. If the client retries with the same ID, the server will skip the charge, preventing duplicate transactions.

```
from flask import Flask, request, jsonify
import sqlite3

app = Flask(__name__)

# Initialize SQLite connection
def get_db():
    conn = sqlite3.connect('database.db')
    return conn

@app.route('/process_payment', methods=['POST'])
def process_payment():
    request_id = request.headers.get("Request-ID")
    db = get_db()
    cursor = db.cursor()

    # Check if request_id already processed
    cursor.execute("SELECT 1 FROM processed_requests WHERE request_id = ?", (request_id,))
    if cursor.fetchone():
        return jsonify({"message": "Duplicate request ignored"}), 200

    # Process payment
    # Here you would include your payment processing logic
    print("Processing payment...")

    # Mark request as processed
    cursor.execute("INSERT INTO processed_requests (request_id) VALUES (?)", (request_id,))
    db.commit()
    return jsonify({"message": "Payment processed successfully"}), 200
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

▼ 2. Database Design Adjustments (Upsert Operation)

Some database operations, such as inserting the same record multiple times, can lead to unintended duplicate entries.

Achieving idempotency in these cases often requires redesigning the database operations to be inherently idempotent.

This can involve using `upsert` operations (which updates a record if it exists or inserts it otherwise) or applying **unique constraints** that prevent duplicates from being added in the first place.

```
INSERT INTO inventory (item_id, stock)  
VALUES (1, 10)  
ON CONFLICT (item_id) DO UPDATE  
SET stock = inventory.stock + EXCLUDED.stock;
```

This SQL statement inserts a new item if it doesn't exist. If it does exist (conflict on `item_id`), it updates the stock by adding the new stock quantity, ensuring the operation remains idempotent.

▼ 3. Idempotency in Messaging Systems

In a messaging system, we can enforce idempotency by storing a log of processed message IDs and checking against it for every incoming message.

```
import java.util.HashSet;  
import java.util.Set;  
  
public class MessageConsumer {  
    private Set<String> processedMessages = new HashSet<>  
    ();  
  
    public void processMessage(String messageId, String messageContent) {  
        if (processedMessages.contains(messageId)) {
```

```

        System.out.println("Duplicate message ignored: " + messageId);
        return;
    }

    // Process the message here
    System.out.println("Processing message: " + messageContent);

    // Add messageId to processed set
    processedMessages.add(messageId);
}
}

```

Each message has a unique `messageId`. Before processing, we check if the `messageId` is already in `processedMessages`. If it is, the message is ignored; otherwise, it's processed and added to the set to avoid duplicates.

▼ 4. Idempotency in HTTP Methods

HTTP defines several methods (verbs) for different types of requests. These methods can be categorized by whether they are idempotent or non-idempotent, influencing how a system handles retries and preventing unintended side effects.

Idempotent Methods:

GET

Retrieves data from a resource. GET requests are inherently idempotent because they only read data and do not alter the server's state.

- **Example:** Accessing a blog post by making a GET request to `/posts/123` will simply retrieve that post, without modifying any server data. Whether you retrieve it once or a thousand times, the post remains unchanged.

PUT

Update or completely replace an existing resource. PUT requests are idempotent because the final state is the same whether the PUT request is executed once or multiple times.

- **Example:** Updating user information by making a PUT request to `/users/45` with updated user details will overwrite the user's data with the new information provided. Executing the same PUT request repeatedly results in the same final user data on the server

DELETE

Removes a resource from the server. DELETE requests are idempotent because deleting a resource that's already been deleted has no further effect.

- **Example:** Deleting an item by making a DELETE request to `/items/678` will remove the item. If you attempt the DELETE request again, it will have no effect since the item no longer exists.

Non-Idempotent Methods:

POST

Creates a new resource on the server. POST requests are non-idempotent because each request usually results in the creation of a new resource.

- **Example:** Creating a new order by making a POST request to `/orders` with order details will generate a new order each time the request is made.

▼ 4. Challenges and Considerations

While idempotency is powerful, it comes with its own set of challenges:

1. **Performance Overhead:** Storing idempotency keys or checking for duplicate operations can add overhead and increase the overall latency.
2. **State Management:** Idempotency often requires maintaining state, which can be challenging in stateless architectures.
3. **Distributed Systems:** Ensuring idempotency across distributed systems can be challenging and may require **distributed locking** or **consensus algorithms**.
4. **Time Window:** How long should idempotency guarantees be maintained? Forever, or for a limited time?

5. **Database Constraints:** Not all operations are idempotent by default; unique constraints or upsert logic may be necessary to avoid duplication.

▼ 5. Best Practices

When implementing idempotency in your system, consider these best practices:

1. **Use Unique Identifiers:** Attach a unique ID (idempotency key) to each request to track and prevent duplicate processing.
2. **Design for Idempotency from the Start:** It's much easier to design for idempotency from the beginning than to add it later.
3. **Implement Retry with Backoff:** When retrying idempotent operations, use an exponential backoff strategy to avoid overwhelming the system.
4. **Employ Idempotent HTTP Methods:** Prefer idempotent methods (GET, PUT, DELETE) for operations that may be retried; design POST with unique identifiers if idempotency is required.
5. **Document Idempotent Operations:** Clearly document which operations are idempotent in your API specifications.
6. **Test Thoroughly:** Implement tests that verify the idempotency of your operations, including edge cases and failure scenarios.
7. **Use Locks or Versioning:** Use locks, optimistic concurrency control, or version numbers to manage simultaneous requests safely.

Idempotency is a powerful concept in distributed systems that can greatly enhance the reliability and fault-tolerance of your systems.

Whether you're designing a distributed database, a payment processing system, or a simple web API, considering idempotency in your design can save you (and your users) from many headaches down the road.

▼ 7. API Design - Best Practises

REST, which is a resource-based architectural style for developing APIs and web services.

"RESTful"—meaning compliant with the principles and constraints of REST architecture.

▼ 1. Learn the basics of HTTP

This will help you make good design choices.

- HTTP has **verbs** (actions or methods): GET, POST, PUT, PATCH and DELETE are most common.
- REST is **resource-oriented** and a resource is represented by an **URI**: `/library/`
- An **endpoint** is the combination of a verb and an URI, example: `GET: /books/`
- An endpoint can be interpreted as an *action performed on a resource*. Example: `POST: /books/` may mean "Create a new book".
- At a high-level, **verbs map to CRUD operations**: `GET` means `Read`, `POST` means `Create`, `PUT` and `PATCH` mean `Update`, and `DELETE` means `Delete`
- A response's status is specified by its **status code 1xx** for **information**, 2xx for **success**, 3xx for **redirection**, 4xx for **client errors** and 5xx for **server errors**.

▼ 2. Do not return plain text

Although this is not imposed or mandated by any REST architectural style, most REST APIs by convention use JSON as the data format.

However, it is not good enough to just return a response body containing a JSON-formatted String. You should still **specify the `Content-Type` header**. It must be set to the value `application/json`.

This is especially important when dealing with **application/programmatic clients** (example, another service/API interacting with your API via the `requests` library in Python)—some of them rely on this header to accurately decode the response.

▼ 3. Do not use verbs in URIs

This is because the **HTTP verbs should be sufficient to accurately describe the action being performed on the resource**.

Example: Let's say that you are providing an endpoint to generate and retrieve a book cover for a book. I will note `:param` a placeholder for an URI parameter (like an ID or a slug). Your first idea might be to create a similar endpoint to this one:

```
GET: /books/:slug/generateBookCover/
```

But, the `GET` method is syntactically sufficient here to say that we are retrieving ("GETting") a book's cover. So, let's just use:

```
GET: /books/:slug/bookCover/
```

Likewise, for an endpoint that creates a new book:

```
# Don't do this
POST: /books/createNewBook/
# Do this
POST: /books/
```

HTTP verbs (*verbalizes*) all the things!

▼ 4. Use plural nouns for resources

Should we use `/book/:id/` (singular) or `/books/:id/` (plural)?

My personal advice is to *use the plural form*.

Why? Because it fits *all types* of endpoints very well.

I can see that `GET /book/2/` is fine. But what about `GET /book/`? Are we GETting the one and only book in the library, couple of them, or *all of them*?

To prevent this kind of ambiguity, **let's be consistent** (💡 Software career advice!) and use plural everywhere:

```
GET: /books/2/
POST: /books/
...
```

▼ 5. Return the error details in the response body

When an API server handles an error, it is convenient (*and recommended*) to return **error details** within the JSON body to **help consumers with debugging**. Even better if you include which fields were affected by the error!

```
{
  "error": "Invalid payload.",
  "detail": {
    "name": "This field is required."
  }
}
```

▼ 6. Pay special attention to HTTP status codes

I feel this one is *pretty important*. If there is one thing you need to remember from this article, this is probably it.

The worst thing your API could do is return an error response with a `200 OK` status code.

It's simply bad semantics. Instead, **return a meaningful HTTP status code** that *accurately* describes the type of error.

Make use of the HTTP status code, and only use the response body to provide error details.

```
HTTP/1.1 400 Bad Request
Content-Type: application/json{
  "error": "Expected at least three items in the list."
}
```

▼ 7. You should use HTTP status codes consistently

Once you've mastered HTTP status codes, you should aim to use them **consistently**.

For example, if you choose that a `POST` endpoint returns a `201 Created` somewhere, use that *same HTTP status code* for every `POST` endpoint.

Why? Because consumers should not have to worry about *which method on which endpoint will return which status code in which circumstances*.

So, be predictable (**consistent**). If you have to stray away from conventions, **document it** somewhere with big signs.

Typically, I stick to the following pattern:

```
GET: 200 OK
PUT: 200 OK
POST: 201 Created
PATCH: 200 OK
DELETE: 204 No Content
```

▼ 8. Do not nest resources

You are probably noticing by now that REST APIs deal with resources. Retrieving a list, or a single instance of a resource is straightforward. But, what happens when you deal with **related resources**?

For example, let's say we want to retrieve the list of books for a particular author — the one with `name=Cagan`. There are basically two options.

The first option would be to **nest** the `books` resource under the `authors` resource, example:

```
GET: /authors/Cagan/books/
```

Some architects recommend this convention because it does indeed represent the **one-to-many relationship** between an author and their books.

But, **it is not clear** anymore what type of resource you are requesting. Is it authors? Is it books? ...

My personal recommendation is to **use query string parameters** to filter the `books` resource directly:

```
GET: /books?author=Cagan
```

And this clearly means: "Get all books for author name Cagan", right? 😊

▼ 9. Handle trailing slashes gracefully

Whether or not URIs should have a trailing slashes `/` is not really a debate. You should simply choose one way or the other (i.e. with or without the trailing slash), stick to it and **gracefully redirect clients if they use the wrong convention**.

(I will admit, I have been guilty of this one myself more than once. 🙈)

Story time! 📖 One day, as I was integrating a REST API into one of my projects, and I kept receiving `HTTP 500 Internal Error` on *every single call*. The endpoint I was using looked something like this:

```
POST: /buckets
```

I was fuming and for the life of me couldn't figure out what the hell I was doing wrong. 🤔

In the end, it turned out that **the server was failing because I was missing a trailing slash!** So, I began using:

```
POST: /buckets/
```

Aaaand everything went fine afterwards. 🧑

💡 **Pro-Tip:** Most web-based frameworks (Angular, React, etc.) have an option to gracefully redirect to the trailed or untrailed version of the URL. Find that option and activate it as early as possible.

▼ 10. Make use of the **querystring** for filtering and pagination

Your consumers may want to retrieve items that fulfill a specific condition, or retrieve them in small amounts at a time to improve performance.

This is exactly what **filtering** and **pagination** are made for.

With **filtering**, consumers can specify parameters (or properties) that the returned items should have.

Pagination allows consumers to retrieve *fractions of the set of data*. The simplest kind of pagination is **page number pagination**, which is determined by a `page` and a `page_size`.

Now, the question is: **How do you incorporate such features in a REST API?**

My answer is: **Use the querystring.**

```
GET: /books?page=1&page_size=10
```

At first, you might think of doing something like this to retrieve a list of *only published* books:

```
GET: /books/published/
```

Design issue: `published` **is not a resource!** Instead, it is a *trait* of the data you are retrieving. That kind of thing should go in the **querystring**.

So in the end, a user could retrieve “the second page of published books containing 20 items” like this:

```
GET: /books?published=true&page=2&page_size=10
```

▼ 11. Know the difference between `401 Unauthorized` and `403 Forbidden`

When handling security errors in a REST API, it is extremely easy to get confused about whether the error relates to **Authentication** or **Authorization** (a.k.a permissions)

- Has the consumer not provided authentication credentials? Was their SSO Token invalid/timed out? 👉 `401 Unauthorized` .
- Was the consumer *correctly authenticated*, but they *don't have the required permissions/proper clearance* to access the resource? 👉 `403 Forbidden` .

▼ 12. Make good use of HTTP `202 Accepted`

I find `202 Accepted` to be a very handy alternative to `201 Created` . It basically means:

I, the server, have understood your request. I have not created the resource (yet), but that is fine.

There are two main scenarios which I find `202 Accepted` to be especially suitable:

- If the resource will be created as a result of future processing — example: After a job/process has finished.
- If the resource already existed in some way, but this should not be interpreted as an error.

▼ 13. Use a web framework specialized in REST APIs

As a last best practice, let's discuss this question: **How do you actually implement best practices in your API?**

Most of the time, you want to create a quick API so that a few services can interact with one another.

Python developers would grab Flask, JavaScript developers would grab Node (Express), and they would implement a few simple routes to handle HTTP requests.

The issue with this approach is that **generally, the framework is not targeted at building REST API servers.**

For example, both Flask and Express are two very versatile frameworks, but they were not *specifically* made to help you build REST APIs.

As a result, you have to take **extra steps** to implement best practices in your API. And most of the times, **laziness or a lack of time mean you will not make the effort** — and leave your consumers with a quirky API.

The solution is simple: **Use the right tool for the job.**

New frameworks have emerged in various languages that are specifically made to build REST APIs. **They help you follow best practices hassle-free without sacrificing productivity.**

In Python, one of the best API framework I've found is Falcon. It's just as simple to use as Flask, nicely fast and perfect for building REST APIs within minutes.

If you're more of a Django type of person, the go-to is the Django REST Framework. It's not as intuitive, but incredibly powerful.

In Node, Restify seems to be a good candidate as well, although I haven't gotten around to trying it yet.

I strongly recommend you give these frameworks a shot. They will help you build beautiful, elegant and well-designed REST APIs.

<https://blog.wahab2.com/api-architecture-best-practices-for-designing-rest-apis-bf907025f5f>

▼ 8. Rate Limiting